

Recursive Sorting Algorithms

To understand the concept of recursion and how it is applied in more efficient sorting algorithms like Merge Sort and Quick Sort, and to analyse their performance characteristics.

Introduction to Recursion

Recursion is a programming technique where a function calls itself in order to solve a problem. A recursive function must have two main components:

Base Case: A condition that stops the recursion from continuing infinitely. This is the simplest form of the problem that can be solved directly.

Recursive Step: The part of the function that calls itself with a smaller version of the problem. Think of it like a set of Russian nesting dolls. Each doll contains a smaller version of itself, until you reach the smallest doll that doesn't contain anything else (the base case).

Examples of Recursion

Let's look at some classic examples of recursive functions.

Factorial

The factorial of a non-negative integer n , denoted by $n!$, is the product of all positive integers less than or equal to n . Example: $5! = 5 * 4 * 3 * 2 * 1$. Note that the factorial of 0 is defined as 1.

- Base Case: $0! = 1$

- Recursive Step: $n! = n \times (n-1)!$ for $n > 0$

def factorial(n):

 #Calculates the factorial of a non-negative integer recursively

 Base Case: if n is 0, the factorial is 1

 if $n == 0$:

 return 1

 # Recursive Step: $n!$ is n times the factorial of $(n-1)$

 else:

 return $n * \text{factorial}(n - 1)$

Example usage:

print(factorial(5)) # Output: 120 ($5 * 4 * 3 * 2 * 1$)

Fibonacci Sequence

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, usually starting with 1 and 1. The sequence goes as follows: 1, 1, 2, 3, 5, 8, 13, 21, Note that the first two terms are fixed, then every other term afterwards is a sum of the previous 2 terms.

- Base Cases: $F(1)=1$ and $F(2)=1$
- Recursive Step: $F(n)=F(n-1)+F(n-2)$ for $n>1$

```
def fibonacci(n):  
    #Calculates the n-th Fibonacci number recursively.  
    # Base Cases:  
    if n <= 1:  
        return n  
    # Recursive Step:  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)  
  
# Example usage:  
print(fibonacci(6)) # Output: 8 (0, 1, 1, 2, 3, 5, 8)
```

Understanding how recursion works is essential for the next two sorting algorithms, as they both employ a divide-and-conquer strategy that is naturally implemented with recursion.

Merge Sort

Merge Sort is a divide-and-conquer sorting algorithm. It works by recursively breaking down the list into sublists until each sublist contains only one element (which is considered sorted). Then, it repeatedly merges these sublists to produce new sorted sublists until there is only one sorted list remaining.

The "merge" step is crucial: it takes two sorted sublists and combines them into a single sorted list.

Think of taking a deck of cards, cutting it in half, cutting each half in half again, and so on, until you have many small piles of one card each. Then, you take pairs of these one-card piles and merge them back together into sorted two-card piles, then merge pairs of two-card piles into sorted four-card piles, and continue until you have one sorted deck.

```
### Code Solution  
def merge_sort(arr):
```

```

#Sorts an array using Merge Sort
if len(arr) <= 1:
    return arr # Base case: a list with 0 or 1 element is sorted

# Divide the array into two halves
mid = len(arr) // 2
left_half = arr[:mid]
right_half = arr[mid:]

# Recursively sort both halves
left_half = merge_sort(left_half)
right_half = merge_sort(right_half)

# Merge the sorted halves
return merge(left_half, right_half)

def merge(left, right):
    """Merges two sorted arrays into a single sorted array."""
    result = []
    i = j = 0

    # Compare elements from left and right and append the smaller to result
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Append remaining elements from left (if any)
    result.extend(left[i:])
    # Append remaining elements from right (if any)
    result.extend(right[j:])

    return result

# Example usage:
my_array = [38, 27, 43, 3, 9, 82, 10]
sorted_array = merge_sort(my_array)

```

```
print(sorted_array) # Output: [3, 9, 10, 27, 38, 43, 82]
'''
```

Big-O Notation

Merge Sort consistently divides the problem in half (logarithmic work for the division part) and then performs linear work to merge the results.

- Worst-Case Scenario: The structure of the algorithm ensures that the work performed at each level of recursion is proportional to the input size. Runtime complexity is $O(n \log n)$
- Average-Case Scenario: The average performance is the same as the worst case. Runtime complexity is $O(n \log n)$
- Best-Case Scenario: Even if the array is already sorted, the divide and merge process still happens, resulting in the same time complexity. Runtime complexity is $O(n \log n)$

Space Complexity: $O(n)$ because the merge function requires extra space to create the result list to store the merged elements.

When to Use Merge Sort

- When a stable sort is required.
- When consistent performance is needed, as it has a guaranteed $O(n \log n)$ time complexity regardless of the input data.
- Suitable for sorting large datasets.
- Can be adapted for external sorting (sorting data that doesn't fit into memory).

Quicksort

Quick Sort is another divide-and-conquer sorting algorithm. It works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then recursively sorted. The process is repeated until the entire array is sorted.

The key step is the 'partition' operation, which rearranges the array such that all elements smaller than the pivot come before it, and all elements greater than the pivot come after it.

Imagine standing in front of a line of people and picking one person (the pivot). You then ask everyone shorter than that person to stand on one side and everyone taller to stand on the other. You then repeat this process independently for the two smaller groups until everyone is in height order.

Code Solution

```
'''
```

```

def quick_sort(arr):
    """Sorts an array using Quick Sort."""
    if len(arr) <= 1:
        return arr # Base case: a list with 0 or 1 element is sorted

    # Choose a pivot (simple implementation uses the last element)
    # More robust implementations might use a random pivot or median-of-three
    pivot = arr[-1]
    less = []
    equal = []
    greater = []

    # Partition the array based on the pivot
    for x in arr:
        if x < pivot:
            less.append(x)
        elif x == pivot:
            equal.append(x)
        else:
            greater.append(x)

    # Recursively sort the less and greater sub-arrays and combine the results
    return quick_sort(less) + equal + quick_sort(greater)

# Example usage:
my_array = [10, 7, 8, 9, 1, 5]
sorted_array = quick_sort(my_array)
print(sorted_array) # Output: [1, 5, 7, 8, 9, 10]

```

Note: The provided Quick Sort implementation is a simple, easy-to-understand version that uses extra space for the `less`, `equal`, and `greater` lists. A more common and space-efficient implementation is an in-place version using partitioning with two pointers, which has a Space Complexity of $O(\log n)$ on average due to the recursion stack.

Big-O Notation

Quick Sort's performance is highly dependent on the choice of the pivot.

- Worst-Case Scenario: The pivot selection consistently results in highly unbalanced partitions (e.g., always picking the smallest or largest element). This happens if the array is already sorted

or reverse sorted when the first or last element is chosen as the pivot. The recursion almost becomes linear. Runtime complexity is $O(n^2)$

- Average-Case Scenario: With good pivot selection (e.g., random pivot, median-of-three), Quick Sort performs very well. Runtime complexity is $O(n \log n)$

- Best-Case Scenario: The pivot selection consistently partitions the array into two roughly equal halves. Runtime complexity is $O(n \log n)$

Space Complexity: The space complexity depends on the implementation. The simple recursive version shown above has a space complexity of $O(n)$ in the worst case due to the creation of new lists in recursive calls. An in-place partitioning implementation has a space complexity of $O(\log n)$ on average and $O(n)$ in the worst case (highly unbalanced recursion).

When to Use Quick Sort

- When fast average-case performance is critical.
- It is often considered one of the fastest sorting algorithms in practice.
- When an in-place sort is preferred (with the more advanced in-place partitioning implementation).